

ANTEPROYECTO DEL TRABAJO DE FIN DE GRADO

INFORMACIÓN GENERAL

Alumno/a	David Muñoz del Valle				
Titulación:	Graduado en Ingeniería del Software				
Tutor/es:	Gabriel Jesús Luque Polo				
Título	Biblioteca extensible de optimización metaheurística en Rust				
Subtítulo <i>(solo si en grupo)</i>					
Título en inglés	Extensible metaheuristic optimization library in Rust				
Subtítulo en inglés <i>(solo si en grupo)</i>					
Trabajo en grupo:	☒ Sí	☐	☐ No	☒ X	
Otros integrantes del grupo:					

INTRODUCCIÓN

Contextualización del problema a resolver. Describir claramente de dónde surge la necesidad de este TFG y el dominio de aplicación. En caso de que el TFG se base en trabajos previos, debe aclararse cuáles son las aportaciones del TFG.

El desarrollo de este Trabajo de Fin de Grado surge ante la escasez de bibliotecas de optimización metaheurística en el ecosistema Rust [3]. Actualmente existen pocas bibliotecas de este tipo, donde podríamos citar MetaheuRUSTics [2]. En general, presentan limitaciones en cuanto a madurez, extensibilidad y variedad de algoritmos disponibles.

Esta situación dificulta el uso de Rust en proyectos que requieren técnicas de optimización metaheurística, a pesar de las ventajas que ofrece el lenguaje en términos de rendimiento y seguridad. Como respuesta a esta necesidad, este TFG propone el diseño y desarrollo de una biblioteca de optimización metaheurística extensible en Rust, que permita ampliar fácilmente el conjunto de algoritmos y adaptarse a distintos tipos de problemas.

El trabajo se apoya en soluciones previas existentes, tanto en Rust como en otros lenguajes, pero aporta como contribución principal una arquitectura modular y orientada a la extensibilidad, orientada a facilitar el desarrollo y la experimentación con metaheurísticas dentro del ecosistema Rust.

OBJETIVOS

Descripción detallada de en qué consistirá el TFG. En caso de que el objeto principal del TFG sea el desarrollo de software, además de los objetivos generales deben describirse sus funcionalidades a alto nivel.

El proyecto pretende ser una forma de ayudar a los desarrolladores a implementar algoritmos de optimización metaheurística en un lenguaje moderno como es Rust. Ya existen bibliotecas de este estilo, pero no hay muchas y están limitadas, la idea es que esta pueda servir a un público mayor, debido a que pretende ser extensible.

El presente Trabajo de Fin de Grado consiste en el diseño y desarrollo de una biblioteca de optimización metaheurística en Rust, cuyo objetivo principal es facilitar a los desarrolladores la implementación y el uso de este tipo de algoritmos en un lenguaje moderno, seguro y eficiente.

Aunque existen algunas bibliotecas de optimización metaheurística en Rust, su número es reducido y presentan limitaciones en cuanto a funcionalidad y capacidad de ampliación. En este contexto, el proyecto propone el desarrollo de una biblioteca orientada a un público amplio, con un diseño modular que permita su extensión mediante la incorporación de nuevos algoritmos, operadores y tipos de problemas, sin necesidad de modificar el núcleo del sistema.

A alto nivel, la biblioteca ofrecerá una arquitectura común para la definición de problemas de optimización y la implementación de distintos algoritmos y problemas junto a los elementos esenciales que permitirán crear otros de forma sencilla. Asimismo, se proporcionarán mecanismos básicos para la configuración de los algoritmos, la ejecución de experimentos y la obtención de resultados, junto con documentación y ejemplos de uso que faciliten su adopción por parte de otros desarrolladores.

ENTREGABLES

Listado de resultados que generará el TFG (aplicaciones, estudios, manuales, etc.)

Software desarrollado: biblioteca Rust

Documentación

Pruebas

MÉTODOS Y FASES DE TRABAJO

METODOLOGÍA:

Descripción de la metodología empleada en el desarrollo del TFG. Especificar cómo se va a desarrollar. Concretar si se trata de alguna metodología existente y, en caso contrario, describir y justificar adecuadamente los métodos que se aplicarán.

Para el desarrollo de la librería se empleará una metodología iterativa e incremental, inspirada en metodologías ágiles y, en particular, en un enfoque similar a SCRUM [6], adaptado al desarrollo individual de una única persona.

El proceso comenzará con una fase inicial de análisis de requisitos y planificación temporal, en la que se definirán los objetivos del trabajo y se dividirá el desarrollo en un conjunto de iteraciones. Cada iteración tendrá una fecha estimada de inicio y finalización, así como un conjunto de funcionalidades a implementar.

Debido a la naturaleza y a las características propias del desarrollo de software, esta metodología se plantea como flexible y adaptable al cambio. Por ello, las iteraciones estarán abiertas a modificaciones en cuanto a duración, alcance o prioridades, siendo completamente normal la aparición de retrasos, adelantos o ajustes en la planificación inicial.

Asimismo, se contempla la posibilidad de que durante el desarrollo surjan requisitos incompletos, inconsistentes o no previstos inicialmente, los cuales se irán refinando y ajustando de forma progresiva conforme avance el proyecto. Este enfoque permite integrar mejoras continuas, realizar refactorizaciones cuando sea necesario y adaptar la arquitectura de la librería para favorecer su extensibilidad y mantenibilidad.

En cada iteración se abordarán las fases de diseño, implementación, pruebas y documentación, garantizando así una validación continua del software desarrollado y una evolución controlada del proyecto hasta su finalización

FASES DE TRABAJO:

Enumeración y breve descripción de las fases de trabajo en las que consistirá el TFG.

1. Estudio

a. Metaheurísticas:

Análisis de algoritmos (Genéticos, Recocido Simulado, búsqueda tabú, etc. [8]) para identificar abstracciones comunes.

b. Lenguaje de programación - Rust:

Aprendizaje del lenguaje de programación utilizado durante el desarrollo del trabajo.

c. Compilador - Cargo:

Aprendizaje y puesta a punto del sistema de compilación y gestor de paquetes oficial de Rust.

d. Crates:

Representan la unidad fundamental de compilación, equivalente a una biblioteca o paquete en otros lenguajes, que contiene código Rust empaquetado. En esta fase se estudiarán los más relevantes de cara a este trabajo.

e. Estado del Arte:

Estudio de bibliotecas existentes en Rust y otros lenguajes para identificar oportunidades de mejora tales como jMetal [1], MetaheuRUSTics [2] u optim.lib [5].

2. Organización, metodología

a. Análisis de requisitos:

Enumerar qué y cómo debe funcionar la biblioteca.

b. División del trabajo en iteraciones:

Adopción de una metodología ágil para la división del trabajo en iteraciones (*Sprints*), permitiendo entregas incrementales de código funcional.

3. Arquitectura

a. Diseño de la arquitectura:

Modelado y decisiones sobre la futura implementación del proyecto.

b. Decisión sobre los patrones de diseño:

Selección de patrones (ej. *Strategy* para operadores, *Builder* para configurar algoritmos) que aprovechen el sistema de Traits de Rust para permitir que el usuario añada sus propias lógicas.

4. Desarrollo

a. Desarrollo de la librería:

Implementación del núcleo de la librería y las metaheurísticas base.

b. Implementación paralela:

Funciones que permitan la ejecución paralela de los algoritmos.

c. Desarrollo de las pruebas:

Desarrollo de tests unitarios y de integración para asegurar la robustez (aprovechando cargo test).

d. Desarrollo de los observables:

Observables que permitan el análisis del funcionamiento del algoritmo, sin ensuciar el código con lógica de telemetría.

e. Desarrollo de ejemplos:

Creación de casos prácticos (ej. Problema del Viajante o Mochila) para demostrar la facilidad de uso y la extensibilidad de la biblioteca.

5. Conclusiones

a. Obtención de métricas:

Pruebas de rendimiento (benchmarks) y comparación de resultados con otras librerías o soluciones conocidas.

b. Análisis de los resultados

c. Obtención de las conclusiones

d. Identificación de las limitaciones:

Reflexión sobre los puntos fuertes del lenguaje y las dificultades encontradas.

6. Documentación y memoria

a. Desarrollo de la documentación:

Se elaborará la documentación del software mediante cargo doc [7], o a través de una web diseñada a medida para ofrecer una exposición más flexible de los contenidos.

b. Desarrollo de licencia, consideraciones legales y configuración del repositorio

Elaboración del documento académico que recoge todo el proceso.

TEMPORIZACIÓN:

La siguiente tabla deberá contener una fila por cada una de las fases enumeradas en la sección anterior. La temporización **no debe incluir las horas dedicadas a la preparación de la presentación y defensa del TFG**.

En caso de tratarse de un **trabajo en grupo**, se añadirá una columna HORAS por cada miembro del equipo. Debe especificarse claramente el número de horas dedicado por cada alumno/a y la suma de horas individual deberá ser también de 296.

FASE	HORAS
	<i>David Muñoz del Valle</i>
1.a. Estudio sobre Metaheurísticas	16
1.b. Estudio sobre el lenguaje de programación (RUST)	25
1.c. Estudio sobre el compilador, Cargo	4
1.d. Estudio sobre los Crates	2
1.e. Estudio sobre el estado del arte	8
2.a. Análisis de los requisitos	7
2.b. División del trabajo en iteraciones	3
3.a. Diseño de la arquitectura	28
3.b. Decisión sobre los patrones de diseño	7
4.a. Desarrollo de la librería	50
4.b. Implementación paralela	15
4.c. Desarrollo de las pruebas	20
4.d. Desarrollo de los observables	10
4.e. Desarrollo de ejemplos	7
5.a. Obtención de métricas	10
5.b. Análisis de los resultados	2
5.c. Obtención de las conclusiones	2
5.d. Identificación de las limitaciones	5
6.a. Desarrollo de la documentación	23
6.b. Desarrollo de licencia, consideraciones legales y configuración del repositorio	5
6.c. Desarrollo de la memoria	47
	296

ENTORNO TECNOLÓGICO

TECNOLOGÍAS EMPLEADAS:

Enumeración de las tecnologías utilizadas (lenguajes de programación, frameworks, sistemas gestores de bases de datos, etc.) en el desarrollo del TFG.

Lenguaje de programación: Rust

Gestión de paquetes y compilación: Cargo

Control de versiones: Git

Repositorio remoto: GitHub

RECURSOS SOFTWARE Y HARDWARE:



Listado de dispositivos (placas de desarrollo, microcontroladores, procesadores, sensores, robots, etc.) o software (IDE, editores, etc.) empleados en el desarrollo del TFG.

Software:

IDE/Editor de código: Visual Studio Code
IDE/Editor de código: Rust Rover
Extensiones Visual Studio Code: Rust Analyzer
Sistema operativo: Arch Linux (64-bit)

Hardware, ordenador personal:

Procesador (CPU):

AMD Ryzen 5 5500U, Radeon
Arquitectura: x86_64 (32-bit y 64-bit)
Núcleos: 6
Hilos: 12 (2 hilos por núcleo)
Frecuencia máxima: 4,06 GHz
Caché: L1 192 KiB, L2 3 MiB, L3 8 MiB
Virtualización: AMD-V

Memoria RAM: 7,1 Gb

Tarjeta gráfica: AMD Lucienne, integrada en CPU

REFERENCIAS

Listado de referencias (libros, páginas web, etc.)

- [1] J.J. Durillo, A.J. Nebro, “[jMetal: a Java Framework for Multi-Objective Optimization](#)”. Advances in Engineering Software 42 (2011) 760-771.
- [2] Shah, A. S. (2025). “MetaheuRUSTics: A comprehensive collection of metaheuristic optimization algorithms implemented in Rust”. URL: <https://github.com/aryashah2k/metaheuRUSTics> Accedido: 07/01/2026
- [3] Rust Foundation, “Documentación oficial del lenguaje Rust”. URL: <https://rust-lang.org/es/> Accedido: 07/01/2026
- [4] Rust Foundation. “The Cargo Book”. URL <https://doc.rust-lang.org/cargo/> Accedido: 07/01/2026
- [5] Keith O’Hara, “Documentación oficial de OptimLib”. URL: <https://optimlib.readthedocs.io/en/latest/> Accedido: 07/01/2026
- [6] Joel Francia Huambachano. “¿Qué es Scrum?” URL: <https://www.scrum.org/resources/blog/que-es-scrum> Accedido: 07/01/2026
- [7] Rust Foundation. “Cargo-doc manpage”. Sección 4.2.6 de The Cargo Book. URL: <https://doc.rust-lang.org/cargo/commands/cargo-doc.html> Accedido: 07/01/2026
- [8] Blum, Christian, and Andrea Roli. "Metaheuristics in combinatorial optimization: Overview and conceptual comparison." *ACM computing surveys (CSUR)* 35.3 (2003): 268-308.

Málaga, 15 de febrero de 2026

Firma tutor/tutora:

Firma cotutor/a:

Firma tutor/a coordinador/a: